Ex 7: Retrieval-Augmented Generation (RAG) using Vector Database and LLM

Learning Objective:

Develop a Python program to integrate a Vector Database with a Large Language Model (LLM) to build a Retrieval-Augmented Generation (RAG) system that answers questions based on external PDF documents such as LinkedIn profiles or research papers.

Steps:

1. Import Required Libraries (OpenAI, Hugging Face Transformers, Vector Database libraries like FAISS/Chroma, PDF processing libraries)
2. Configure OpenAI API key
   (This key should be personal. Do not share with anyone).
3. Load PDF Documents
   Use LinkedIn profile PDFs or research paper PDFs as the dataset.
4. Extract Text from PDFs
   Convert PDF content into raw text using suitable libraries.
5. Split Text into Chunks
   Divide extracted text into smaller chunks for efficient retrieval.
6. Generate Embeddings
   Convert text chunks into vector embeddings using embedding models.
7. Store Embeddings in Vector Database
   Use FAISS or Chroma to store and index embeddings.
8. Initialize the LLM
   Load OpenAI or Hugging Face model for generating responses.
9. Create Retrieval Function
   Retrieve relevant document chunks from the vector database based on user query.

10. Build RAG Pipeline
    Combine retrieved context with user query and pass it to the LLM to generate answers.
11. Provide User Query
    Ask questions related to the content of the PDF documents.
12. Generate Responses
    Use retrieved information + LLM to produce accurate answers.
13. Display the Output
    Present retrieved context and generated answer.

CODE :

```
Import os
from dotenv import load_dotenv
import google.generativeai as genai
import faiss
import numpy as np
from sentence_transformers import SentenceTransformer
from PyPDF2 import PdfReader
from sklearn.preprocessing import normalize
from google.colab import userdata
load_dotenv()
ge na i. co nfigure (ap i_ke y= ( 'AIzaS yDq BW F NA_ g8cZ LJ yjDr QPjq w_pr PRp4r ho '
)
llm = genai.GenerativeModel("gemini-2.5-flash")
def load_pdf(file_path):
    reader = PdfReader(file_path)
    text = ""
```

```
    forpage in reader.pages:
        text += page.extract_text() or ""
    return text

pdf_path = "/content/VD002640-RESUME..pdf"
pdf_text = load_pdf(pdf_path)
print("\nPDF Loaded. Length:", len(pdf_text))
def chunk_text(text, chunk_size=300):
    return [text[i:i+chunk_size] for i in range(0, len(text), chunk_size)]
chunks = chunk_text(pdf_text)
model = SentenceTransformer('all-MiniLM-L6-v2'
embeddings = model.encode(chunks)
embeddings = normalize(np.array(embeddings).astype('float32'))
dimension = embeddings.shape[1]
index = faiss.IndexFlatIP(dimension)
index.add(embeddings)
def retrieve(query, k=3):
    q_vec = model.encode([query])
    q_vec = normalize(q_vec).astype('float32')
    scores, indices = index.search(q_vec, k)
    results = []
    for i, idx in enumerate(indices[0]):
        results.append({
            "text": chunks[idx],
            "score": float(scores[0][i])
        })
    return results
def rag_answer(query):
    retrieved = retrieve(query)
```

```
    context_text = "\n\n".join([r["text"] for r in retrieved])
    prompt = f"""
Use the context below to answer the question.
Context:
{context_text}
Question:
{query
Answer clearly and concisely:
"""
    response = llm.generate_content(prompt)
    return retrieved, response.text
query = "What skills are mentioned in this resume?"
retrieved_chunks, answer = rag_answer(query
print("\n===== RETRIEVED CHUNKS =====\n")
for i, r in enumerate(retrieved_chunks):
    print(f"Chunk {i+1} (Score: {r['score']:.4f}):\n{r['text']}\n")
print("\n===== FINAL ANSWER =====\n")
print(answer)
```

231501173

OUTPUT:

```
...
PDF Loaded. Length: 2149
Loading weights: 100% [▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰] 103/103 [00:00<00:00, 516.32it/s, Materializing param=pooler.dense.weight]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                     | Status     |  |
------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you e

===== RETRIEVED CHUNKS =====

Chunk 1 (Score: 0.3040):
d
software development. Passionate about exploring emerging technologies, deriving insights from
practical solutions to real-world problems.
EDUCATION
B.Tech – Artificial Intelligence & Machine Learning
 2023 – 2027 (Pursuing)
 Rajalakshmi Engineering College, Chennai
Diploma – El

Chunk 2 (Score: 0.2961):
ectrical and Electronics Engineering
```

```
...
B.Tech – Artificial Intelligence & Machine Learning
 2023 – 2027 (Pursuing)
 Rajalakshmi Engineering College, Chennai
Diploma – El

Chunk 2 (Score: 0.2961):
ectrical and Electronics Engineering
 Completed 2024
 Ramakrishna Polytechnic College, Chennai
Higher Secondary Education (SSLC)
 Completed 2021
 Bharath Metric Higher Secondary School, Ponneri
TECHNICAL SKILLS
■ Python
■ SQL
■ MS Excel
■ Power BI
■ Tableau
■ Data Visualization
■ HTML / CSS / JavaSc

Chunk 3 (Score: 0.2272):
ript
■ Java (Basic)
■ IoT / Arduino
PROJECTS
HR Analytics Dashboard
 Power BI
 Built an interactive HR analytics dashboard to track employee metrics, attrition trends, and
Retail & Sales Analytics
```

```
===== FINAL ANSWER =====

The skills mentioned in this resume are:
*   Python
*   SQL
*   MS Excel
*   Power BI
*   Tableau
*   Data Visualization
*   HTML / CSS / JavaScript
*   Java (Basic)
*   IoT / Arduino
```

Learning Outcome:

Students will be able to build a Retrieval-Augmented Generation (RAG) system by integrating vector databases with LLMs, enabling accurate question answering over external PDF documents such as LinkedIn profiles or research papers.